

CV Tasks, Convolutions, Pooling

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



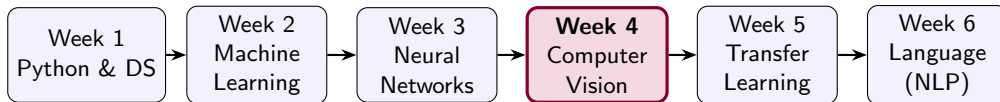
جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 4, Day 22, Lecture

10 October 2026

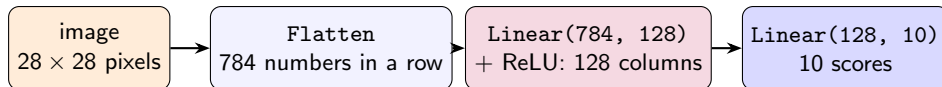
1. Big Picture and Motivation
2. Learning Outcomes
3. Images and the Three Vision Tasks
4. Convolution
5. Padding, Stride and Output Shapes
6. Pooling
7. The First CNN: Conv, ReLU, Pool, Flatten, Linear
8. Olympiad Practice
9. Limitations and What Comes Next
10. Summary
11. References



Today: Day 22 of 42

- ▶ **Week 3:** a network is layers of neurons, each a line plus a bend (Day 15), trained by the four-move loop with a validation pass (Day 16) and made reliable by Day 17's dials. We fed it images by **flattening** them into rows.
- ▶ **Today:** a layer built for images, the **convolution**. A CNN is the Week 3 network with this layer in front; everything else (loss, loop, dials) stays.
- ▶ **This week:** tomorrow, CNN architectures and augmentation; Monday, detection and segmentation; then three self-study days and the Friday competition.

How We Did It Before: Flatten, Then a Network



$$784 \cdot 128 + 128 + 128 \cdot 10 + 10 = 101,770 \text{ dials (Day 16's MNIST network)}$$

- ▶ Day 16's recipe for digits: cut the picture into one long row, then a fully connected network reads the row like any table.
- ▶ It works. On the 10,000 MNIST test digits, a one-hidden-layer network of 300 units gets **4.7%** of them wrong (470 digits), a 300-then-100 network **3.05%**, and a plain line **12%** (LeCun et al., 1998).
- ▶ Every hidden unit is a **template** the size of the whole image (Day 15's picture of the hidden weights): 784 weights, one per pixel position.

Three problems, all caused by Flatten

- ▶ **Every weight is glued to one pixel position.** Move the digit two pixels to the right and the ink lands on pixels whose weights learned something else. Nothing in Linear says “weight 412 and weight 414 should agree”: a stroke in the corner and the same stroke in the centre are two separate things to learn.
- ▶ **Neighbours are forgotten.** After flattening, pixel 412 and pixel 413 (side by side) are as unrelated to the network as pixel 412 and pixel 700. The row has no idea which numbers were next to each other.
- ▶ **It does not scale.** A 224×224 colour photo has $224 \cdot 224 \cdot 3 = 150,528$ numbers; one hidden layer of 256 units needs $150,528 \cdot 256 + 256 = 38,535,424$ dials before the first ReLU.

The number that settled it

On MNIST the fully connected networks above stopped between 3% and 5% error. A convolutional network with only about 2,600 parameters (LeNet-1) reached **1.7%**, and LeNet-5 reached **0.95%** (LeCun et al., 1998): fewer dials, far fewer mistakes.

In one sentence

A **convolution** is a tiny neuron, a 3×3 window of 9 weights and a bias, applied at every position of the image with the *same* 10 dials; its outputs form a new image called a feature map.

- ▶ **Local:** each output looks at 9 neighbouring pixels, so neighbours are kept together.
- ▶ **Shared:** the same dials at every position, so a stroke is learned once and found anywhere. 10 dials replace 784.
- ▶ **Stacked:** convolution, bend, pool, repeat; the first layer finds edges, later layers combine them into parts and objects. The last layer is Week 3's Linear: a CNN is still a neural network, trained with the same loop.

Where you will meet it

Every image task in an AI Olympiad (classify photos, find objects, outline regions) starts from convolutional layers, and this week's three tasks share them as their first stage. Day 25 deepens kernels, padding and shapes; tomorrow builds the famous architectures from today's pieces.

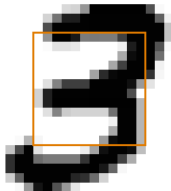
By the end of this session, you will be able to:

- ▶ **Explain** what an image is to a computer (a tensor of channels $\times$ height $\times$ width) and tell the three vision tasks apart by their input, output and metric.
- ▶ **Compute** a convolution by hand: slide a 3×3 kernel, sum the products, and say what the kernel detects.
- ▶ **Show** that a convolutional layer is a neural-network layer whose neurons are local and share their weights, and count its parameters.
- ▶ **Predict** the output shape of a convolution from the kernel size, padding and stride, and check it in PyTorch.
- ▶ **Apply** max pooling by hand and say what it keeps and what it throws away.
- ▶ **Build** a small CNN from Conv2d, ReLU, MaxPool2d, Flatten and Linear, trace its shapes, and train it with the Week 3 loop.

Computer Vision: **Images and the Three Vision Tasks**

An Image Is a Grid of Numbers

an MNIST digit, label 3: 28 x 28 pixels



the orange window: 12 x 12 numbers, 0 = white, 255 = black

0	109	252	252	230	132	133	132	132	189	252	252
0	4	29	29	24	0	0	0	0	14	226	252
0	0	0	0	0	0	0	0	0	85	243	252
0	0	0	0	0	0	0	0	0	88	189	252
0	0	0	0	0	0	0	0	0	91	212	247
0	32	125	193	193	193	253	252	252	252	238	102
45	222	252	252	252	252	253	252	252	252	177	0
45	223	253	253	253	253	255	253	253	253	253	74
0	31	123	52	44	44	44	44	143	252	252	74
0	0	0	0	0	0	0	0	0	15	252	74
0	0	0	0	0	0	0	0	0	86	252	74
9	0	0	0	0	0	0	0	93	242	252	74

Source: MNIST (LeCun et al., 1998) via OpenML mnist_784, image 7; scripts/day22.cv_figures.py.

- ▶ A grey image is a table of **height** × **width** numbers, one per **pixel**: here $28 \times 28 = 784$ numbers from 0 (no ink) to 255 (full ink). Day 16 flattened exactly this table into a row.
- ▶ The digit is *where the big numbers are*. The stroke of the 3 is a run of 252s with 0s on both sides; that neighbourhood is what Flatten forgets.

Colour: Three Channels

the photo: 344 X 335 pixels X 3 channels



red channel: 344 X 335 numbers, 0 to 255



green channel: 344 X 335 numbers, 0 to 255



blue channel: 344 X 335 numbers, 0 to 255



Source: Penn-Fudan Database for Pedestrian Detection and Segmentation (Wang, Shi, Song and Shen, 2007), image FudanPed00005; scripts/day22_cv_figures.py.

- ▶ A colour pixel is **three** numbers: how much red, green and blue. Pixel (row 100, column 200) of this photo is (176, 197, 30): much red and green, little blue, a yellow-green leaf.
- ▶ So a colour image is three grids, the **channels**, stacked: $3 \times 344 \times 335 = 345,720$ numbers. PyTorch orders the shape as (**channels, height, width**); a grey image is (1, 28, 28).
- ▶ A batch adds a first axis (Day 19): 64 colour photos of 32×32 are a tensor of shape (64, 3, 32, 32). ToTensor() also rescales 0 to 255 into 0 to 1.

```
from PIL import Image
from torchvision import transforms
img = Image.open("FudanPed00005.png").convert("RGB")    # a PIL image
x = transforms.ToTensor()(img)                            # (3, 344, 335), values 0..1
print(x.shape, x.min().item(), x.max().item())
print(x[:, 100, 200])                                    # one pixel: its three channels
batch = x.unsqueeze(0)                                    # (1, 3, 344, 335): a batch of one
```

- ▶ **(C, H, W)** is the order every layer of today expects. The first thing to print in any vision task is `x.shape`; the second is `x.min()`, `x.max()` to know whether the values are 0 to 255 or 0 to 1.
- ▶ A greyscale image loaded this way has $C = 1$; a PNG with transparency has $C = 4$. `.convert("RGB")` makes every file three channels.

The Three Tasks on One Photo

classification: one label for the whole image
'pedestrian' (yes), 344 x 335 x 3 numbers in



detection: a box and a label per object
2 boxes, each (x, y, width, height) + 'person'



segmentation: a label for every pixel
115,240 pixels, each 'person' or 'background'



Source: Penn-Fudan Database for Pedestrian Detection and Segmentation (Wang et al., 2007), image FudanPed00005 with its human-drawn masks; the boxes are the smallest rectangles around each mask; `scripts/day22.cv_figures.py`.

- Same input, three outputs of different **shape**: one label, a list of boxes, a grid of labels. The shape of the output decides the loss, the metric and the last layer; the first layers (convolutions, today) are shared by all three.

Task 1: Image Classification

Definition

Image classification: one image in, **one label** out of k classes out. Day 08's classification task with an image as the row.

- **Output:** k scores, softmax, argmax (Day 16). **Loss:** cross-entropy. **Metric:** accuracy, or macro F1 when the classes are unbalanced (Day 09).
- **Data:** a folder per class, or a CSV of file names and labels; the labels are the only thing to predict.
- **The classic sets:** MNIST (digits, 10 classes), CIFAR-10 (tiny photos, 10 classes; Krizhevsky, 2009), ImageNet (1.2 million photos, 1,000 classes; Deng et al., 2009), the one on the right.

Image Classification

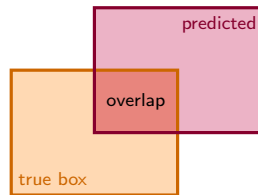


Source: Krizhevsky, Sutskever and Hinton (2012), Fig. 4 (left): eight ImageNet test images and the network's five most likely labels; the correct one is red.

Definition

Object detection: one image in, a **list** out; each entry is a class label and a **bounding box** (x, y, w, h) : the box's top-left corner, width and height in pixels.

- ▶ The list has no fixed length: two pedestrians give two entries, an empty street none. That is why the output is not “ k scores” and why detection needs its own last layers (Day 24: YOLO).
- ▶ **Metric:** a predicted box is right when it has the right label *and* overlaps the true box enough. The overlap is measured by **IoU**; the whole list is scored by **mAP** (mean average precision), both deepened on Days 24 and 26.



$$\text{IoU} = \frac{\text{area of the overlap}}{\text{area covered by either box}}$$

Boxes $4 \times 3 = 12$ each, overlap $2 \times 1.5 = 3$:
 $\text{IoU} = \frac{3}{12+12-3} = 0.14$. A match usually needs $\text{IoU} \geq 0.5$.

Definition

Segmentation: one image in, a **label for every pixel** out: a grid of class numbers the same height and width as the image, called the **mask**.

- **Semantic** segmentation labels pixels by class (“person”, “road”); **instance** segmentation also separates the two people, as the figure’s colours do.
- **Output:** k scores *per pixel*, a tensor (k, H, W) ; the mask is the argmax at each pixel. **Loss:** cross-entropy averaged over pixels. **Metric:** IoU between predicted and true mask, class by class (Dice is its cousin; Day 26).
- **Network:** convolutions down to a small map, then back up to full size: U-Net (Day 24).

	0	1	2	3	4
0	0	0	0	0	0
1	0	1	1	0	0
2	0	1	1	1	0
3	0	0	1	1	0
4	0	0	0	0	0

A 5×5 mask: 0 = background, 1 = person. Eight pixels are “person”; the metric compares this grid with the predicted one, pixel by pixel.

Task	In	Out	Loss	Metric	This week
Classification	$(3, H, W)$	k scores	cross-entropy	accuracy, F1	Days 22, 23, 25
Detection	$(3, H, W)$	boxes + labels, any number	box + class losses	IoU, mAP	Days 24, 26
Segmentation	$(3, H, W)$	(k, H, W) , scores per pixel	per-pixel cross-entropy	IoU, Dice	Days 24, 26

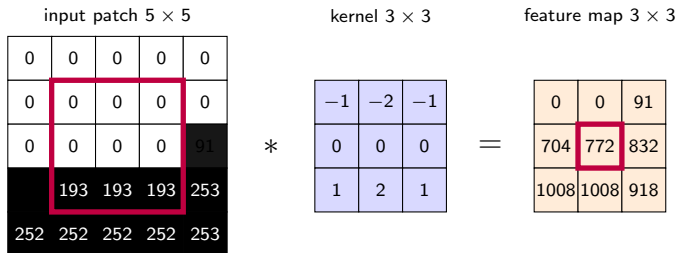
- ▶ Read a task statement for its **output shape** and its **metric**; they tell you which of the three it is and which last layer and loss to use. The submission file follows: one label per image, one line per box, or one mask per image.
- ▶ All three start the same way: convolutional layers turning pixels into feature maps. That shared beginning is the rest of today.

Convolution: **A Small Window That Slides**

Definition: A Kernel Slides Over the Image

Definition

A **convolution** slides a small grid of weights, the **kernel** (or filter), over the image. At each position it multiplies the kernel by the pixels under it, adds the products (plus a bias), and writes one number. The grid of these numbers is the **feature map**.



Kernel: 3×3 , so 9 weights + 1 bias = 10 dials, the same at every position. **Window:** the 3×3 patch under the kernel now (purple, rows 11 to 13 and columns 11 to 13 of the digit). **Feature map:** one number per window position; a 5×5 patch has 3×3 positions. The purple window gives the purple cell, 772: next frame.

$$y[i, j] = b + \sum_{u=0}^2 \sum_{v=0}^2 K[u, v] \cdot X[i + u, j + v]$$

X : the image; K : the 3×3 kernel; b : its bias (0 below); $y[i, j]$: the output whose window starts at row i , column j . Nine products, one sum, one number.

The purple window of the digit

$$X_{\text{win}} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 193 & 193 & 193 \end{pmatrix} \text{ (white above, ink below), } K_{\text{horizontal}} = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}, \quad K_{\text{vertical}} = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

$$y_{\text{horizontal}} = (-1)(0) + (-2)(0) + (-1)(0) + 0 + 0 + 0 + (1)(193) + (2)(193) + (1)(193) = \mathbf{772}$$

$$y_{\text{vertical}} = (-1)(193) + (0)(193) + (1)(193) + (\text{six zeros}) = \mathbf{0}$$

A kernel is a **pattern detector**: large where the window looks like its pattern (bright rows below dark rows), near 0 where it does not.

The horizontal-edge kernel at all nine positions of the 5×5 patch:

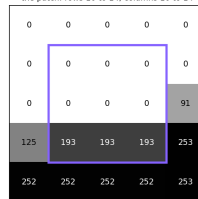
$$y = \begin{pmatrix} 0 & 0 & 91 \\ 704 & 772 & 832 \\ 1008 & 1008 & 918 \end{pmatrix}$$

- ▶ Top row: windows of white only, 0.
- ▶ Middle and bottom rows: windows across the top of the stroke, large. The feature map is a **map of where the pattern is**.
- ▶ The vertical kernel gives $\begin{pmatrix} 0 & 0 & 91 \\ 68 & 0 & 242 \\ 136 & 0 & 212 \end{pmatrix}$:
small, except at the right where the 91 and 253 start a left-to-right change.

the digit and a 5×5 patch



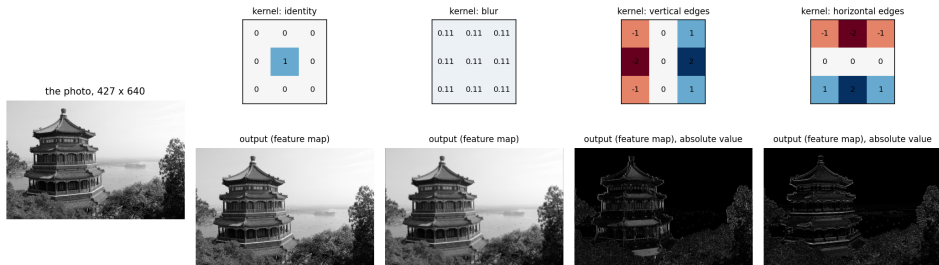
the patch: rows 10 to 14, columns 10 to 14



Source: MNIST image 7 via OpenML; orange: the 5×5 patch; purple: the window of the worked example; `scripts/day22_cv_figures.py`.

5×5 in, 3×3 out: a 3×3 kernel loses one pixel on every side. Padding fixes that.

Hand-Made Kernels on a Real Photo



Source: scikit-learn's sample photograph china.jpg (427 × 640, averaged to one grey channel), convolved with `torch.nn.functional.conv2d`; `scripts/day22_cv_figures.py`.

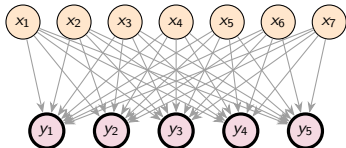
- **Identity** copies the pixel; **blur** averages the nine neighbours; the **edge** kernels (Sobel, 1968) light up where brightness changes left to right, or top to bottom.
- Before deep learning, people typed kernels like these by hand. Today's layer lets gradient descent choose the nine numbers.

```
import torch, torch.nn as nn, torch.nn.functional as F
x = torch.tensor(digit, dtype=torch.float32)[None, None] # (1, 1, 28, 28)
K = torch.tensor([[[-1., -2., -1.], [0., 0., 0.], [1., 2., 1.]]])[None, None]
y = F.conv2d(x, K) # a fixed kernel: y is (1, 1, 26, 26)
print(y[0, 0, 11, 11].item()) # 772.0, the worked example
layer = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3)
print(layer.weight.shape) # (16, 1, 3, 3): 16 learnable kernels
print(layer(x).shape) # (1, 16, 26, 26): 16 feature maps
```

- ▶ `F.conv2d` applies a kernel you give it; `nn.Conv2d` owns its kernels as **weights with slopes**, like `nn.Linear`: started at random, turned by `loss.backward()` and `opt.step()`.
- ▶ The four axes are always (batch, channels, height, width); a kernel's are (out channels, in channels, height, width).

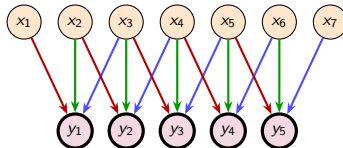
A Convolution Is a Layer of Neurons

fully connected (Linear(7, 5)): $7 \cdot 5 + 5 = 40$ dials



every output reads every input, each arrow its own weight

convolution (kernel size 3): $3 + 1 = 4$ dials



every output reads 3 neighbours with the *same* w_1, w_2, w_3

- ▶ Each y_j is a Day 15 neuron: $y_j = f(w_1x_j + w_2x_{j+1} + w_3x_{j+2} + b)$. New: it reads only its **neighbours** (local), and all five neurons use the **same** dials (shared). A 3×3 kernel on an image is this picture in two dimensions.
- ▶ Everything from Week 3 applies: ReLU after it, the chain rule through it, the same four moves. A CNN is a neural network.

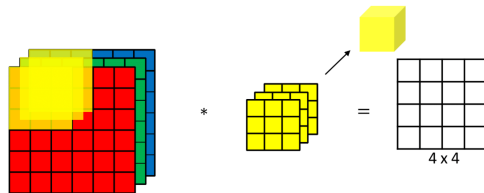
Parameters of a convolutional layer

$C_{out} \cdot (C_{in} \cdot k \cdot k) + C_{out}$: one $k \times k$ kernel per (input channel, output channel) pair, plus one bias per output channel. The image size does not appear.

Input	Layer	Dials	Outputs
$1 \times 28 \times 28$ digit	Linear(784, 128)	$784 \cdot 128 + 128 = 100,480$	128 numbers
$1 \times 28 \times 28$ digit	Conv2d(1, 16, 3)	$16 \cdot (1 \cdot 9) + 16 = 160$	$16 \times 26 \times 26$ map
$3 \times 32 \times 32$ photo	Linear(3072, 16)	$3072 \cdot 16 + 16 = 49,168$	16 numbers
$3 \times 32 \times 32$ photo	Conv2d(3, 16, 3)	$16 \cdot (3 \cdot 9) + 16 = 448$	$16 \times 30 \times 30$ map
$3 \times 224 \times 224$ photo	Linear(150528, 16)	2,408,464	16 numbers
$3 \times 224 \times 224$ photo	Conv2d(3, 16, 3)	448	$16 \times 222 \times 222$ map

- The convolution's 448 dials stay 448 when the photo grows 49 times; the Linear layer's grow with it. And the convolution keeps a *map* (where things are), the Linear layer keeps 16 numbers.

- ▶ A colour input has $C_{in} = 3$ channels, so a kernel is $3 \times 3 \times 3$: 27 weights and one bias. The window is a small cube; the sum runs over the three channels too and still gives **one** number per position.
- ▶ One kernel gives one feature map. $C_{out} = 16$ kernels give 16 maps stacked: output $(16, H', W')$. The next layer has $C_{in} = 16$ and kernels of $16 \times 3 \times 3$.
- ▶ So **channels are the columns of an image**: red, green, blue at the input; “vertical edge here”, “blob of green here” after a layer.



Source: Andrew Ng, *Deep Learning Specialization*, Course 4 (Convolutional Neural Networks), Coursera: a $6 \times 6 \times 3$ input, a $3 \times 3 \times 3$ kernel, a 4×4 feature map.

Check

$\text{Conv2d}(16, 32, 3)$: $32 \cdot (16 \cdot 9) + 32 = 4,640$ dial; 32 output channels.

Learned kernels

Trained on photos, the first layer's kernels become edge detectors at many angles, colour-contrast detectors and blobs, close to the hand-made ones (Krizhevsky et al., 2012, Fig. 3: AlexNet's 96 learned 11×11 kernels). Nobody types them; gradient descent finds them because the later layers need them.

- ▶ Deeper layers combine edges into corners and textures, then parts (an eye, a wheel), then objects (Zeiler and Fergus, 2014). Stacking is how a 3×3 window ends up seeing a whole cat.

Why sharing wins

Move the digit three pixels right. The windows holding the stroke sit three pixels right too, with the *same* dials, so the feature map moves with it and the stroke is still found. The flattened network had to learn the shifted stroke as a new thing.

- ▶ **Translation equivariance:** shift the input, the feature map shifts the same way. LeCun et al. (1998) built it in with three ideas, *local receptive fields*, *shared weights*, *subsampling*; the third is pooling.
- ▶ A kernel is learned once for all positions: 10 dials, not 784.

Convolution: **Padding, Stride and Output Shapes**

Definition

Padding p adds p rows and columns of zeros around the image before the kernel slides. With $p = 1$ a 3×3 kernel has a window centred on every pixel, border included, and the output keeps the input's size.

- ▶ Without padding every 3×3 layer shrinks the map by 2: $28 \rightarrow 26 \rightarrow 24 \rightarrow \dots$; ten layers eat 20 pixels, and border pixels are seen by fewer windows than the centre.
- ▶ $p = (k - 1)/2$ keeps the size: padding=1 for $k = 3$, padding=2 for $k = 5$ ("same" padding).
- ▶ The zeros are not ink: the kernel sees an artificial edge there, usually harmless.

5 $\times$ 5 image, $p = 1$: a 7 $\times$ 7 grid

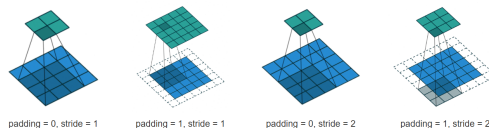
0	0	0	0	0	0	0
0						0
0						0
0						0
0						0
0						0
0	0	0	0	0	0	0

the first window is centred on pixel (0, 0)

Definition

The **stride** s is how far the window moves between two positions. $s = 1$: every position. $s = 2$: every second row and column, so the output is about half as wide and half as tall.

- ▶ Stride 2 is a cheap way to shrink the map: a quarter of the positions, a quarter of the arithmetic, and the next layer's windows cover twice as much of the original image.
- ▶ Modern networks (ResNet, tomorrow) shrink with stride-2 convolutions; the classic way is the pooling layer, next section. Both are “subsampling” in LeCun’s three ideas.



Source: Dumoulin and Visin (2016), *A guide to convolution arithmetic for deep learning*, arXiv:1603.07285: a 3×3 kernel on a 5×5 input (blue) and the output (green) for four settings.

Left to right: $p = 0, s = 1$ gives 3×3 ;
 $p = 1, s = 1$ gives 5×5 ; $p = 0, s = 2$ gives
 2×2 ; $p = 1, s = 2$ gives 3×3 .

$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

- n_{in} : input height (or width); k : kernel size; p : padding; s : stride; $\lfloor \cdot \rfloor$: round down. The count of window positions: the padded size minus the kernel, divided by the jump, plus the first position.

n_{in}	k	p	s	arithmetic	n_{out}
28	3	0	1	$\lfloor (28 + 0 - 3)/1 \rfloor + 1 = 25 + 1$	26
28	3	1	1	$\lfloor (28 + 2 - 3)/1 \rfloor + 1 = 27 + 1$	28 (same)
28	5	2	1	$\lfloor (28 + 4 - 5)/1 \rfloor + 1 = 27 + 1$	28 (same)
28	3	1	2	$\lfloor (28 + 2 - 3)/2 \rfloor + 1 = \lfloor 13.5 \rfloor + 1$	14 (half)
28	5	0	2	$\lfloor (28 + 0 - 5)/2 \rfloor + 1 = \lfloor 11.5 \rfloor + 1$	12

Check yourself: a 32×32 photo, $k = 3$, $p = 1$, $s = 2$: $\lfloor 31/2 \rfloor + 1 = 16$; then $k = 3$, $p = 0$, $s = 1$: $16 - 2 = 14$. Channels never enter the formula; the number of kernels sets them.

```
x = torch.zeros(1, 1, 28, 28)          # a fake batch of one digit
for k, p, s in [(3, 0, 1), (3, 1, 1), (3, 1, 2), (5, 0, 2)]:
    y = nn.Conv2d(1, 8, kernel_size=k, padding=p, stride=s)(x)
    print(k, p, s, tuple(y.shape))
# 3 0 1 (1, 8, 26, 26)          3 1 1 (1, 8, 28, 28)
# 3 1 2 (1, 8, 14, 14)          5 0 2 (1, 8, 12, 12)
```

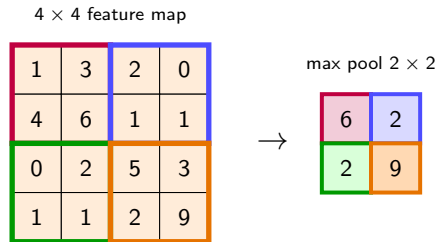
- ▶ Before training anything, push a zero tensor of the right shape through the layers and print what comes out. The most common CNN bug is a Linear layer sized for the wrong map (“mat1 and mat2 shapes cannot be multiplied”); this habit catches it in a second. Day 25 is the deep dive on shape debugging.
- ▶ Note the 8: the number of kernels sets the output channels, independent of k , p and s .

Pooling: **Keep the Strongest, Shrink the Map**

Definition

Max pooling slides a 2×2 window with stride 2 over a feature map and keeps the **largest** number in each window. The map becomes half as tall and half as wide; the channels are untouched, each pooled on its own.

- **Average pooling** keeps the mean instead. Max is the usual choice inside a CNN; average appears at the end of modern networks (tomorrow).
- **No dials:** a fixed rule, like Flatten. Windows do not overlap: a quarter of the numbers survive.



$$\max(1, 3, 4, 6) = 6, \max(2, 0, 1, 1) = 2, \\ \max(0, 2, 1, 1) = 2, \max(5, 3, 2, 9) = 9.$$

Average pooling: $\begin{pmatrix} 3.5 & 1 \\ 1 & 4.75 \end{pmatrix}$, e.g.
 $(1 + 3 + 4 + 6)/4 = 3.5$.

- ▶ **It answers “is the pattern here?”** A feature map says how strongly the pattern appeared at each position; after max pooling it says whether it appeared *somewhere in each 2×2 block*. The detection survives, the exact pixel is dropped.
- ▶ **Small shifts stop mattering.** Move the 772 one pixel right and the pooled value is still 772 while it stays in its block. Flattened networks had no such tolerance.
- ▶ **It shrinks the work.** $16 \times 28 \times 28 = 12,544$ numbers become $16 \times 14 \times 14 = 3,136$; the next convolution runs on a quarter of the positions, and each of its windows covers 6×6 original pixels; after two poolings, 12×12 . Small kernels end up seeing big things.
- ▶ **It makes the final Linear small.** $32 \times 7 \times 7 = 1,568$ numbers into the classifier instead of 25,088.

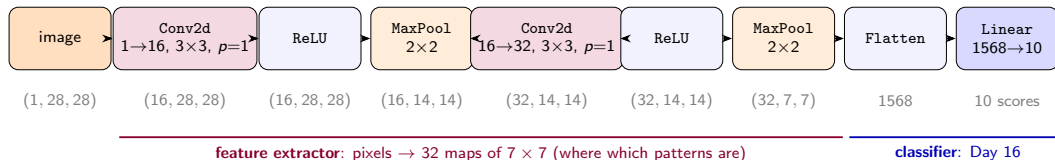
What it throws away

Position inside the block, and three of every four numbers. Fine for “is there a cat” (classification); a problem for “which pixels are the cat” (segmentation), which is why Day 24’s U-Net climbs back up to full size.

```
pool = nn.MaxPool2d(kernel_size=2) # stride defaults to the kernel size: 2
fmap = torch.tensor([[1., 3., 2., 0.], [4., 6., 1., 1.],
                    [0., 2., 5., 3.], [1., 1., 2., 9.]]) [None, None]
print(pool(fmap)[0, 0]) # [[6., 2.], [2., 9.]]
print(nn.AvgPool2d(2)(fmap)[0, 0]) # [[3.50, 1.00], [1.00, 4.75]]
print(pool(torch.zeros(1, 16, 28, 28)).shape) # (1, 16, 14, 14)
print(pool(torch.zeros(1, 16, 15, 15)).shape) # (1, 16, 7, 7): odd row
        dropped
```

- ▶ Same four axes as the convolution; the output size follows the same formula with $k = s = 2, p = 0$: $\lfloor (28 - 2)/2 \rfloor + 1 = 14$, $\lfloor (15 - 2)/2 \rfloor + 1 = 7$. An odd map loses its last row and column; count on it when sizing the Linear layer.
- ▶ `sum(p.numel() for p in pool.parameters())` is 0: nothing to train.

The First CNN: **Conv, ReLU, Pool, Flatten, Linear**



- ▶ Read the shapes left to right: channels grow ($1 \rightarrow 16 \rightarrow 32$), the map shrinks ($28 \rightarrow 14 \rightarrow 7$). Each convolution keeps the size (padding 1); each pooling halves it. $32 \cdot 7 \cdot 7 = 1,568$ is the number the Linear layer must be told.
- ▶ The right-hand end is exactly Week 3: a flattened vector, a linear layer, 10 scores, cross-entropy. Only the start is new, and it is made of today's three pieces.

```

cnn = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),      # (16, 14,
        14)
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),    # (32, 7,
        7)
    nn.Flatten(), nn.Linear(32 * 7 * 7, 10))
h = torch.zeros(1, 1, 28, 28)      # the shape habit, layer by layer
for layer in cnn:
    h = layer(h); print(layer.__class__.__name__, tuple(h.shape))
print(sum(p.numel() for p in cnn.parameters()))      # 20490

```

- ▶ The same `nn.Sequential` as Day 15; `Conv2d` and `MaxPool2d` are just two more layer types. `Flatten` moves from the front (Day 16) to just before the classifier.
- ▶ The loop prints `Conv2d (1, 16, 28, 28), ..., Flatten (1, 1568), Linear (1, 10)`: if `Linear` were told 1,000 instead of 1,568, the error would show here, before any training.

Layer	Formula	Dials	Output
Conv2d(1, 16, 3)	$16 \cdot (1 \cdot 9) + 16$	160	(16, 28, 28)
ReLU, MaxPool2d(2)	nothing to learn	0	(16, 14, 14)
Conv2d(16, 32, 3)	$32 \cdot (16 \cdot 9) + 32$	4,640	(32, 14, 14)
ReLU, MaxPool2d(2), Flatten	nothing to learn	0	1568
Linear(1568, 10)	$1568 \cdot 10 + 10$	15,690	10
the CNN		20,490	
Day 16's 784-128-10 network	$784 \cdot 128 + 128 + 128 \cdot 10 + 10$	101,770	

- ▶ A fifth of the dials of the flattened network, and three quarters of them sit in the last Linear. The two convolutions, which do the seeing, cost 4,800 dials together.
- ▶ Dials are not arithmetic: the 160-dial convolution runs its 10-dial neuron at $16 \cdot 784$ positions, so a CNN epoch is *slower* than an MLP epoch with more dials. That is the price of sharing, and the reason vision work wants a GPU.

```
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(cnn.parameters(), lr=1e-3)
for epoch in range(epochs):
    cnn.train()
    for xb, yb in train_loader:
        flattened
        loss = loss_fn(cnn(xb), yb)
        opt.zero_grad(); loss.backward()
        convolutions too
        opt.step()
    cnn.eval()
    on Day 16
```

xb: (64, 1, 28, 28), not
1. predict 2. loss
3. gradient, through the
4. update: the kernels move
validation pass, checkpoint, as

- ▶ Not one line of Day 16's loop changes: the DataLoader hands over (64, 1, 28, 28) batches, the model happens to start with convolutions, `loss.backward()` applies the chain rule through them because a convolution is sums and products like everything else.
- ▶ Day 17's dials transfer as they are: Adam or SGD with momentum, a learning-rate schedule, BatchNorm after a convolution (BatchNorm2d), dropout before the Linear, early stopping on the validation metric.

Model on MNIST (10,000 test digits)	Kind	Parameters	Test error
linear classifier (one line per digit)	flattened	7,850	12.0%
one hidden layer, 300 units	flattened	about 240,000	4.7%
two hidden layers, 300 then 100	flattened	about 270,000	3.05%
LeNet-1	convolutional	about 2,600	1.7%
LeNet-4	convolutional	about 17,000	1.1%
LeNet-5	convolutional	about 60,000	0.95%
LeNet-5, trained with shifted and distorted copies	convolutional	about 60,000	0.8%

Source: LeCun, Bottou, Bengio and Haffner (1998), Section III and Fig. 9; the parameter counts of the fully connected networks are $784 \cdot 300 + 300 + 300 \cdot 10 + 10$ and $784 \cdot 300 + 300 + 300 \cdot 100 + 100 + 100 \cdot 10 + 10$.

- ▶ The smallest CNN, with a hundredth of the dials, makes a third of the mistakes of the best flattened network. More dials did not help the flattened networks (4.7% with 300 units, 4.5% with 1,000); the right *shape* of layer did.
- ▶ The same story at scale: on ImageNet 2012 (1,000 classes), the convolutional AlexNet reached 15.3% top-5 error against 26.2% for the best non-convolutional entry (Krizhevsky et al., 2012). Tomorrow's architectures are its descendants.

How People Size a Small CNN

Rule of thumb	Source	Catch
Blocks of Conv2d 3×3 (padding 1), ReLU, MaxPool2d(2); two or three blocks for 28 to 64 pixel images	LeCun et al. 1998; Simonyan and Zisserman 2015 (VGG: only 3×3 kernels)	each block halves the map: stop before 1×1
Double the channels after every pool: 16, 32, 64 (or 32, 64, 128)	Simonyan and Zisserman 2015; Goodfellow et al. 2016, ch. 9	the Linear layer is re-sized by hand: print the shapes
One Linear to the classes after Flatten; dropout before it on small data	Krizhevsky et al. 2012; Day 17	no ReLU after the last layer: cross-entropy wants raw scores
Adam 10^{-3} (or SGD, momentum 0.9), batch 64, 5 to 20 epochs, early stopping on validation	Day 17; Day 20's recipe card	an epoch is slow on CPU: time one before planning a sweep
From scratch only on small images with thousands of examples; on photos start from a pretrained CNN	Day 20's recipe card; Week 5	from scratch on a few hundred photos memorises them

Starting points, used the Day 18 way: begin here, read the train-against-validation curves, grow or regularise, keep the smallest shape within noise of the best.

Convolution: **Olympiad Practice**

Task

Write `simple_conv2d(input_matrix, kernel, padding, stride)`: pad the 2-D NumPy array with padding rows and columns of zeros, slide the kernel with the given stride, and return the feature map as a 2-D array (sum of products at every position, no bias).

```
X = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]])
K = np.array([[1, 0], [-1, 1]])
simple_conv2d(X, K, padding=1, stride=2)
# expected: [[ 1.  1. -4.]
#            [ 9.  7. -4.]
#            [ 0. 14. 16.]
```

- ▶ **Question 1:** before coding, what output shape does the formula give for a 4×4 input, $k = 2$, $p = 1$, $s = 2$?
- ▶ **Question 2:** why are the two numbers in the right column negative, when every pixel of X is positive?

Adapted from the Deep-ML open problem bank, problem 41 "Simple Convolutional 2D Layer" (github.com/Open-Deep-ML/DML-OpenProblem, contributed by Drogozov Dima);

the expected output is the bank's own example.

Recognise: the definition frame as two loops over output positions; padding, stride and the output shape from their frames.

```
def simple_conv2d(input_matrix, kernel, padding, stride):
    Xp = np.pad(input_matrix, padding)           # zeros around the border
    kh, kw = kernel.shape
    h = (Xp.shape[0] - kh) // stride + 1         # the output-shape formula
    w = (Xp.shape[1] - kw) // stride + 1
    out = np.zeros((h, w))
    for i in range(h):
        for j in range(w):
            window = Xp[i * stride:i * stride + kh, j * stride:j * stride + kw]
            out[i, j] = np.sum(window * kernel)   # sum of products
    return out
```

1. $\lfloor (4 + 2 - 2)/2 \rfloor + 1 = 3$: a 3×3 map, which the expected output confirms before a single product is computed.
2. The right column's windows hang over the zero padding: at $(0, 2)$ the window is $\begin{pmatrix} 0 & 0 \\ 4 & 0 \end{pmatrix}$, so $1 \cdot 0 + 0 \cdot 0 + (-1) \cdot 4 + 1 \cdot 0 = -4$: the -1 meets a real pixel, the $+1$ entries meet zeros.
3. **Check:** `F.conv2d` on X and K as $(1, 1, 4, 4)$ and $(1, 1, 2, 2)$ tensors with `padding=1`, `stride=2` prints the same nine numbers: no flipping of the kernel in PyTorch.

Convolution: **Limitations & What Comes Next**

- ▶ **A kernel sees neighbours only.** Things far apart in the image meet only after many layers of stacking and pooling; Week 6's attention is built for such relations.
- ▶ **Shift-tolerant, not rotation- or scale-tolerant.** A digit turned by 30° or twice as big is a new pattern for every kernel. The fix is to train on turned and resized copies: data augmentation, tomorrow.
- ▶ **Epochs are expensive.** The 10-dial neuron runs at every position of every channel; a 224×224 photo through a real network is billions of products. Without a GPU, keep images small and runs short.
- ▶ **Small data still overfits.** 20,490 dials memorise a few hundred photos. From scratch needs thousands of images per class; otherwise start from kernels learned elsewhere (Week 5).

The common mistake

A Linear layer sized for the wrong map, or an image handed over as (height, width, channels) instead of (channels, height, width). Both show up before training if you push a zero tensor through the layers and print every shape.

- ▶ **Tomorrow (Day 23), CNNs, architectures, augmentation:** LeNet, VGG and ResNet as today's blocks stacked deeper and wider, BatchNorm2d and skip connections, and the augmentations (flips, crops, rotations) that make a CNN tolerate what sharing alone does not.
- ▶ **Monday (Day 24), detection and segmentation:** the same feature extractor with different ends: YOLO's boxes and U-Net's per-pixel masks, scored with IoU and mAP.
- ▶ **Days 25 to 27, self-study:** kernels, padding and shape debugging in depth, then image classification end to end; YOLO, U-Net and the metrics with numbers; then the Olympiad habits for vision tasks.
- ▶ **Week 5:** kernels learned on a million photos, reused on yours: transfer learning.

Convolution: **Summary**

- ▶ **An image** is a tensor (channels, height, width) of numbers 0 to 255. **Classification** returns one label, **detection** a list of boxes and labels, **segmentation** a label per pixel; the output shape fixes the loss and the metric (accuracy or F1; IoU and mAP; IoU and Dice).
- ▶ **A convolution** slides a 3×3 kernel, sums nine products and a bias, and writes a feature map: 772 where the window held a horizontal edge, 0 where it did not. Trained kernels become edge and colour detectors.
- ▶ **It is a layer of neurons** that read only their neighbours and share their weights: $C_{out}(C_{in}k^2) + C_{out}$ parameters whatever the image size; a stroke learned once is found anywhere.
- ▶ **Padding and stride** set the output shape, $\lfloor (n + 2p - k)/s \rfloor + 1$: padding 1 keeps a 3×3 layer the same size, stride 2 halves it. Print the shapes before training.
- ▶ **Max pooling** keeps the largest of each 2×2 block: half the height and width, no weights, tolerance to small shifts.
- ▶ **The first CNN** is conv, ReLU, pool, twice, then Flatten and Linear: 20,490 weights against 101,770, Week 3's loop unchanged. On MNIST, LeNet-5 made 0.95% errors, the best flattened network 3.05%.

Convolution: **References & Resources**

- [1] LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86(11). LeNet-5, the three ideas (local receptive fields, shared weights, subsampling) and the MNIST error-rate comparison quoted in the motivation and the CNN section.
- [2] Krizhevsky, A., Sutskever, I. and Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS* 25. AlexNet; Fig. 3 (learned first-layer kernels) and Fig. 4 (classification examples, reused in the classification frame); the 15.3% against 26.2% top-5 result.
- [3] Zeiler, M. and Fergus, R. (2014). Visualizing and understanding convolutional networks. *ECCV*. What the layers of a trained CNN respond to: edges, textures, parts, objects.
- [4] Dumoulin, V. and Visin, F. (2016). *A guide to convolution arithmetic for deep learning*. arXiv:1603.07285. The padding and stride figure and the output-shape formula.
- [5] Simonyan, K. and Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *ICLR*. VGG: stacks of 3×3 convolutions with channel widths doubling after each pooling.
- [6] Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*, MIT Press, chapter 9 (Convolutional Networks): sparse interactions, parameter sharing, equivariance, pooling. <https://www.deeplearningbook.org>

- [7] Sobel, I. and Feldman, G. (1968). A 3×3 isotropic gradient operator for image processing. Presented at the Stanford Artificial Intelligence Project. The two edge kernels of the photo frame.
- [8] Wang, L., Shi, J., Song, G. and Shen, I. (2007). Object detection combining recognition and segmentation. ACCV. The Penn-Fudan Database for Pedestrian Detection and Segmentation (170 images with human-drawn masks), image FudanPed00005 in the task and channel frames. https://www.cis.upenn.edu/~jshi/ped_html/
- [9] LeCun, Y., Cortes, C. and Burges, C. *The MNIST database of handwritten digits*; loaded through OpenML (mnist_784) for the pixel-grid and worked-example frames.
- [10] Krizhevsky, A. (2009). *Learning multiple layers of features from tiny images*. Technical report, University of Toronto. CIFAR-10. Deng, J. et al. (2009). ImageNet: a large-scale hierarchical image database. CVPR.
- [11] Ng, A. *Deep Learning Specialization*, Course 4: Convolutional Neural Networks, Coursera. The convolution-over-volumes figure in the channels frame.
- [12] Deep-ML open problem bank, problem 41 *Simple Convolutional 2D Layer* (contributed by Drogozov Dima). Adapted for the practice task. <https://github.com/Open-Deep-ML/DML-OpenProblem>
- [13] PyTorch documentation: `torch.nn.Conv2d`, `torch.nn.MaxPool2d`, `torch.nn.functional.conv2d`; torchvision `transforms.ToTensor`. <https://pytorch.org/docs>

- [14] `scikit-learn, sklearn.datasets.load_sample_image("china.jpg")`: the photograph of the kernels frame.
- [15] Figures without another source: `slides-builder/scripts/day22_cv_figures.py` (deterministic illustrations from the public data above; it also prints every number of the worked examples). No model was trained for this deck; every comparison number is quoted from the papers above.